



UNIVERSIDAD CATÓLICA DEL NORTE
FACULTAD DE INGENIERÍA Y CIENCIAS GEOLÓGICAS
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

EVALUACIÓN COMPLEMENTARIA 3

Vicente Castro

Mayckol Olivares

Profesor: Nicolas Henríquez

Asignatura: Programación Científica

Fecha: 21/07/2026

1. Índice

Contenido

1. Índice	2
Introducción	3
Desarrollo.....	4
Sitio web elegido	4
Descripción del flujo de scraping	4
Modelo de datos.....	5
Explicación de las consultas	6
Obstáculos encontrados	7
Reflexión final.....	8
Capturas de pantalla.....	9
Navegador Selenium en ejecución	9
Terminal – progreso del scraping y consultas	9
Base de datos data.db – tabla categoría.....	10
Base de datos data.db – tabla libro.....	10
Salida de ruff check	10
Sección de uso de inteligencia artificial	11
Uso 1	11
Uso 2	11
Uso 3	11
Uso 4	11
Uso 5	12
Uso 6	12

Introducción

La obtención de información estructurada desde sitios web es una etapa clave en muchos flujos de trabajo con datos, sobre todo cuando esa información no está disponible en un formato descargable directo. A diferencia de un scraper simple basado en requests y BeautifulSoup, que solo descarga y parsea HTML estático, trabajar con sitios que dependen de interacción o carga dinámica exige simular el comportamiento de un usuario real: navegar, esperar a que los elementos aparezcan y hacer clic sobre componentes concretos del DOM.

Este proyecto implementa un proceso completo de *web scraping* utilizando **Selenium WebDriver** sobre el sitio Books to Scrape, extrayendo más de 50 ítems —en este caso, 1000 libros— con múltiples campos de información cada uno: título, precio, valoración, disponibilidad, categoría y URL de detalle. El programa parte desde la página de inicio, navega hasta las distintas categorías del catálogo y recorre la paginación de cada una hasta agotar los resultados disponibles.

Se puso especial énfasis en dos aspectos técnicos exigidos por el proyecto: el uso de esperas explícitas (WebDriverWait junto con expected_conditions) en lugar de pausas fijas como time.sleep(), y la tolerancia a fallos parciales, de modo que un error al procesar un ítem individual no interrumpa la ejecución completa del scraper. Los datos extraídos se persisten en una base de datos relacional construida con **SQLModel**, sobre la cual se implementaron cuatro consultas de verificación para validar la integridad de la información y explorar patrones básicos del catálogo obtenido.

Este informe documenta el sitio elegido y su justificación, el flujo de navegación implementado, el modelo de datos definido, los resultados obtenidos mediante las consultas y una reflexión final sobre las particularidades de usar Selenium frente a otras herramientas de extracción.

Desarrollo

Sitio web elegido

Para este proyecto se utilizó el sitio sugerido por el docente, Books to Scrape (<https://books.toscrape.com>), un catálogo ficticio diseñado específicamente para practicar web scraping de forma legal y sin restricciones técnicas: no presenta CAPTCHA, bloqueos por IP ni contenido oculto tras JavaScript. El sitio expone más de 1.000 libros distribuidos en 50 categorías temáticas, cada uno con título, precio, valoración por estrellas, disponibilidad y categoría, cumpliendo ampliamente los requisitos mínimos de ítems y campos establecidos para el proyecto.

Se optó por este sitio en lugar de uno propio principalmente porque su estructura HTML es estable y predecible, lo que permitió centrar el esfuerzo de desarrollo en el correcto uso de Selenium —esperas explícitas, manejo de paginación, tolerancia a fallos— en lugar de invertir tiempo adicional en sortear medidas anti-scraping de un sitio real. No se presentaron problemas técnicos que forzaran un cambio de sitio durante el desarrollo.

Descripción del flujo de scraping

El programa inicia el navegador mediante `webdriver.Chrome`, gestionado automáticamente a través de `webdriver_manager`, y carga la página de inicio del sitio (<https://books.toscrape.com>). Desde ahí, utiliza `WebDriverWait` junto con `expected_conditions.presence_of_all_elements_located` para esperar a que el menú lateral de categorías (`.side_categories` ul li ul li a) esté disponible, y extrae de cada enlace su nombre y su URL.

Con la lista de categorías obtenida, el scraper visita cada una de forma secuencial mediante `driver.get()`. Para cada categoría, espera la presencia de al menos un elemento `article.product_pod` —la tarjeta que representa a cada libro en el listado— antes de comenzar a extraer datos, evitando así intentar leer contenido que aún no ha terminado de cargar.

Por cada libro encontrado en la página se extraen seis campos: **título** (atributo `title` del enlace `h3 a`), **precio** (texto de `.price_color`, convertido a `float` tras remover el símbolo de libra), **disponibilidad** (booleano derivado de si el texto de `.availability` contiene "In stock"), **valoración** (entero de 1 a 5, obtenido a partir de la clase CSS del elemento `.star-rating`, mediante un diccionario de conversión de palabras en inglés a números), **categoría** (heredada del contexto de navegación) y **url_detalle** (enlace a la ficha individual del libro). Cada libro se procesa dentro de un bloque `try/except`, de modo que si algún campo no puede extraerse, el error se registra en consola y el scraper continúa con el siguiente ítem sin interrumpir la ejecución global.

Una vez procesados todos los libros visibles en una página, el scraper busca el enlace "next" (li.next a); si existe, hace clic sobre él mediante Selenium y vuelve a esperar la presencia de article.product_pod antes de continuar extrayendo. Si el enlace no existe —porque se llegó a la última página de la categoría—, la excepción producida por find_element se captura y el bucle de paginación se corta, pasando a la siguiente categoría. Este proceso se repite hasta recorrer las 50 categorías del sitio, tras lo cual el navegador se cierra limpiamente con driver.quit().

Cabe destacar que en ningún punto del código se utiliza time.sleep(): toda la sincronización se resuelve mediante WebDriverWait y expected_conditions, cumpliendo estrictamente el requisito de esperas explícitas del proyecto.

Modelo de datos

El modelo de datos se compone de dos entidades relacionadas mediante SQLAlchemy, siguiendo una relación uno-a-muchos entre categorías y libros.

Entidad Categoría

Campo	Tipo	Restricciones
Id	Int	Clave primaria, autoincremental
Nombre	Str	Único, indexado
libros	list[Libro]	Relación inversa (no persistida como columna)

Entidad Libro

Campo	Tipo	Restricciones
Id	Int	Clave primaria, autoincremental
titulo	Str	No nulo
precio	float	No nulo
valoracion	int	No nulo (1 a 5)
disponible	bool	No nulo
url_detalle	str	Único — usado como clave de idempotencia
categoria_id	Optional[int]	Clave foránea hacia Categoría.id

La relación entre ambas entidades se declara mediante `Relationship(back_populates=...)` en ambos sentidos, permitiendo navegar desde una categoría hacia todos sus libros y viceversa. La idempotencia del proceso de carga se garantiza en `seed.py` de dos formas: antes de crear una categoría se verifica si ya existe una con el mismo nombre, y antes de insertar un libro se verifica si ya existe uno con la misma `url_detalle` — si existe, se omite, evitando duplicados en ejecuciones sucesivas del seed.

Explicación de las consultas

- **Consulta 1 — `total_items()`**

Cuenta el total de registros en la tabla Libro mediante `func.count()`. Sirve como verificación básica de que el proceso de scraping y persistencia se completó correctamente. Sobre los datos reales, el resultado fue **1000 libros**, coincidiendo exactamente con el total esperado según la paginación del sitio (1000 libros en 50 categorías).

- **Consulta 2 — `cantidad_por_categoria()`**

Realiza un JOIN entre Libro y Categoría, agrupa por categoría y ordena de mayor a menor cantidad de libros. La categoría con más libros fue **Default** con 152, seguida de **Nonfiction** (110) y **Sequential Art** (75); en el otro extremo, varias categorías (Crime, Erotica, Cultural, Novels, entre otras) tienen un solo libro. Esto refleja la distribución real y desbalanceada del catálogo del sitio, donde algunas categorías temáticas concentran la mayoría de los títulos.

- **Consulta 3 — `top_10_mas_caros()`**

Ordena los libros por precio de forma descendente y retorna los 10 primeros. Se eligió el precio como criterio porque es el campo numérico más representativo del catálogo para comparar ítems entre sí. El libro más caro fue *The Perfect Play (Play by Play #1)* a £59.99, seguido de *Last One Home* (£59.98) y *Civilization and Its Discontents* (£59.95); los diez libros más caros se concentran en un rango muy estrecho, entre £59.45 y £59.99.

- **Consulta 4 — `estadisticas_precios()`**

Calcula promedio, mínimo y máximo del campo precio de forma global usando `func.avg`, `func.min` y `func.max`. El precio promedio del catálogo fue **£35.07**, con un mínimo de **£10.00** y un máximo de **£59.99**, lo que indica una dispersión amplia de precios a lo largo de las 1000 observaciones.

Obstáculos encontrados

Durante el desarrollo del scraper surgieron algunos puntos que requirieron especial atención, principalmente relacionados con la forma en que el sitio expone ciertos datos:

- **Codificación de la valoración por estrellas.** El sitio no expone la valoración de cada libro como un número directo, sino como una palabra en inglés ("One", "Two", "Three"...), incluida dentro de la clase CSS del elemento `.star-rating` (por ejemplo, `star-rating Three`). Fue necesario inspeccionar el HTML para descubrir este comportamiento y construir un diccionario de conversión (`{"One": 1, "Two": 2, ...}`) que tradujera el texto a un valor entero utilizable en el modelo de datos.
- **Detección del fin de la paginación.** El sitio no indica explícitamente cuántas páginas tiene cada categoría, por lo que la estrategia adoptada fue intentar localizar el enlace "next" (`li.next a`) en cada iteración: mientras el elemento existe, se hace clic y se continúa extrayendo; cuando ya no existe —porque se llegó a la última página—, `find_element` lanza una excepción que se captura para cortar el bucle. Esta solución evitó tener que calcular de antemano el número de páginas por categoría, aunque exigió confirmar que la ausencia del botón "next" es efectivamente la señal de término, y no un error de otro tipo.
- **Formato del precio.** El precio de cada libro se muestra en el sitio como texto con el símbolo de libra esterlina (por ejemplo, £45.17), por lo que fue necesario limpiar el string —removiendo el símbolo— antes de convertirlo a float y poder almacenarlo como un campo numérico en la base de datos.

Ninguno de estos puntos detuvo el desarrollo del scraper de forma significativa, pero sí requirieron inspección directa del HTML del sitio y ajustes puntuales en la lógica de extracción antes de llegar a la implementación final.

Reflexión final

El uso de Selenium para este proyecto implicó una complejidad considerablemente mayor que la de un scraper basado en requests y BeautifulSoup. Mientras que este último enfoque se limita a descargar el HTML estático y parsearlo, Selenium requiere levantar una instancia real de navegador, gestionar tiempos de carga, y sincronizar cada interacción con el estado actual del DOM mediante WebDriverWait. Esto se tradujo en un tiempo de ejecución considerablemente mayor —recorrer las 50 categorías y su paginación toma varios minutos, frente a los segundos que tomaría el mismo volumen de datos con requests—, pero fue la elección adecuada para practicar el manejo de sitios que, en casos reales, sí dependen de JavaScript para renderizar su contenido.

La principal ventaja de Selenium es justamente esa capacidad de interactuar con el sitio como lo haría un usuario real: hacer clic en enlaces, esperar cargas asíncronas y navegar por flujos que no son accesibles mediante una simple petición HTTP. La limitación más evidente es el costo en rendimiento y recursos: cada categoría y cada página de paginación implica una recarga completa del navegador, lo que hace que el proceso sea significativamente más lento y sensible a pequeños cambios en la estructura del sitio —un cambio en un selector CSS puede romper el scraper completo.

Como mejoras futuras, sería posible paralelizar el recorrido de categorías utilizando múltiples instancias de WebDriver o herramientas como Selenium Grid, reduciendo el tiempo total de ejecución. También sería valioso incorporar manejo de proxies y rotación de user-agents para dar mayor robustez frente a sitios con medidas anti-scraping más agresivas que las de Books to Scrape, así como exportar los datos extraídos a formatos adicionales (CSV, JSON) además de la base de datos SQLite.

Capturas de pantalla

Navegador Selenium en ejecución

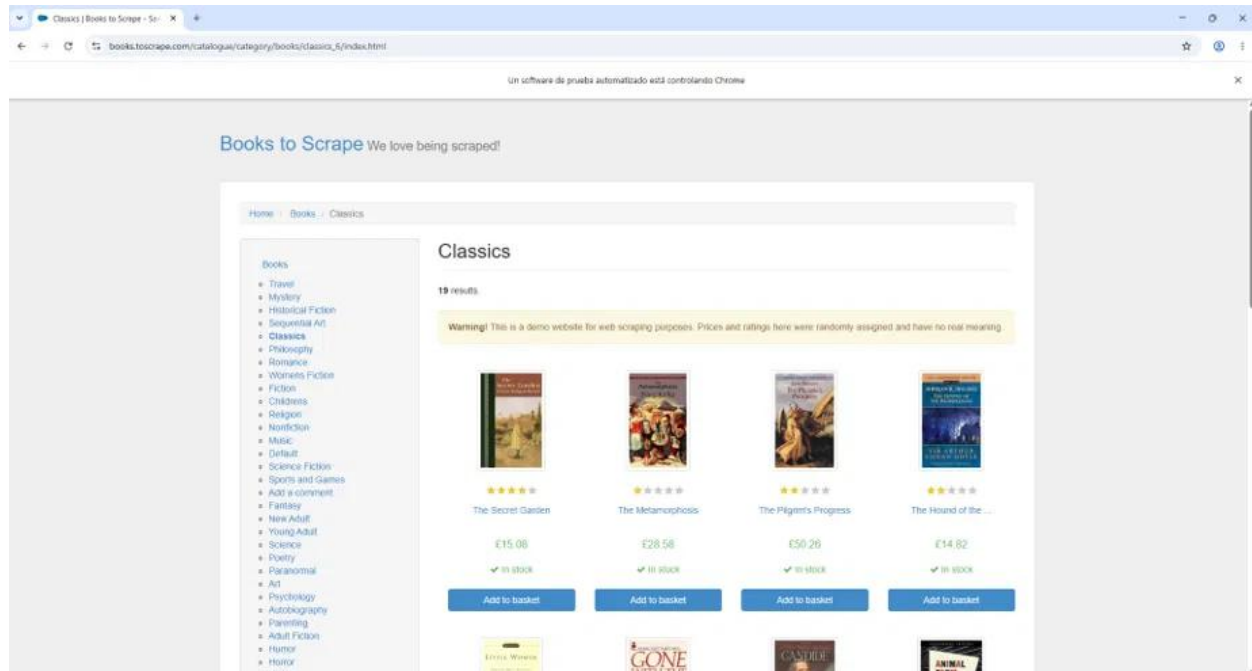


Figura 1. Selenium controlando Chrome sobre la categoría "Classics" de Books to Scrape.

Terminal - progreso del scraping y consultas

```
=====
BOOKS TO SCRAPE
=====
1. Total de libros
2. Cantidad por categoría
3. Top 10 libros más caros
4. Estadísticas de precios
5. Salir

Seleccione una opción: 1

Total de libros: 1000

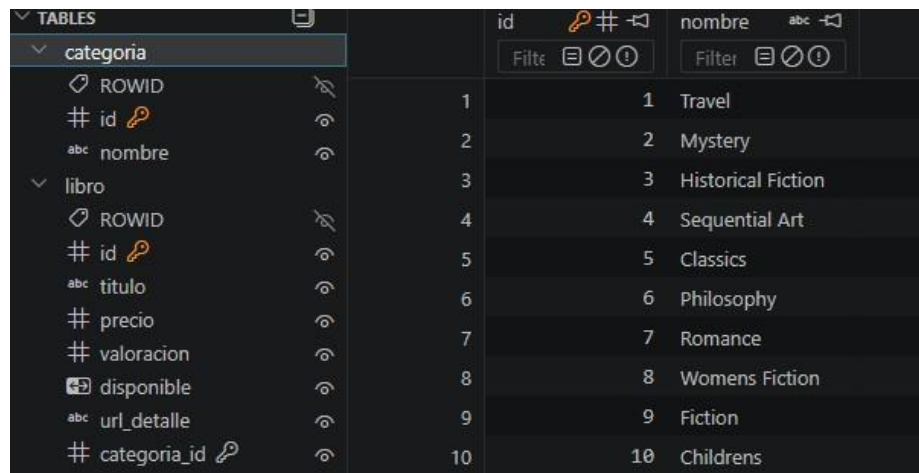
=====
BOOKS TO SCRAPE
=====
1. Total de libros
2. Cantidad por categoría
3. Top 10 libros más caros
4. Estadísticas de precios
5. Salir

Seleccione una opción: 2

Default          152
Nonfiction        110
Sequential Art   75
Add a comment    67
Fiction           65
Young Adult      54
Fantasy          48
Romance          35
Mystery          32
Food and Drink   30
Childrens        29
Historical Fiction 26
Poetry           19
Classics         19
History          18
```

Figura 2. Resultados de consultas luego de scraping.

Base de datos data.db - tabla categoría

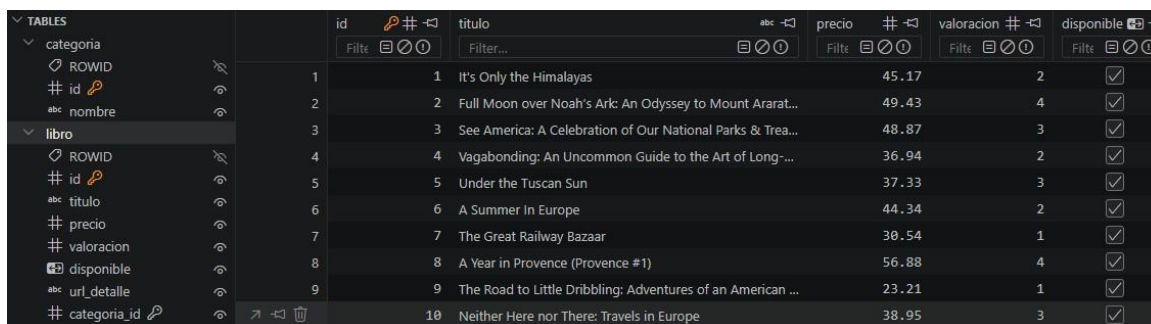


The screenshot shows a SQLite client interface with a sidebar on the left listing tables: 'categoria' and 'libro'. The 'categoria' table is selected, and its structure and data are displayed in the main pane. The table has columns: ROWID, id (primary key), nombre, and categoria_id (foreign key to libro). The data shows 10 categories.

id	nombre
1	Travel
2	Mystery
3	Historical Fiction
4	Sequential Art
5	Classics
6	Philosophy
7	Romance
8	Womens Fiction
9	Fiction
10	Childrens

Figura 3. Tabla categoría en data.db, vista con un cliente SQLite.

Base de datos data.db - tabla libro

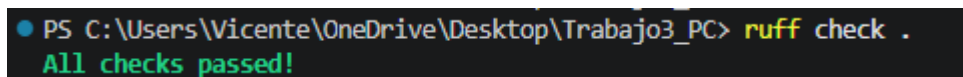


The screenshot shows a SQLite client interface with a sidebar on the left listing tables: 'categoria' and 'libro'. The 'libro' table is selected, and its structure and data are displayed in the main pane. The table has columns: ROWID, id (primary key), titulo, precio, valoracion, disponible, url_detalle, and categoria_id (foreign key to categoria). The data shows 10 books.

id	titulo	precio	valoracion	disponible
1	It's Only the Himalayas	45.17	2	✓
2	Full Moon over Noah's Ark: An Odyssey to Mount Ararat...	49.43	4	✓
3	See America: A Celebration of Our National Parks & Treas...	48.87	3	✓
4	Vagabonding: An Uncommon Guide to the Art of Long...	36.94	2	✓
5	Under the Tuscan Sun	37.33	3	✓
6	A Summer in Europe	44.34	2	✓
7	The Great Railway Bazaar	30.54	1	✓
8	A Year in Provence (Provence #1)	56.88	4	✓
9	The Road to Little Dribbling: Adventures of an American ...	23.21	1	✓
10	Neither Here nor There: Travels in Europe	38.95	3	✓

Figura 4. Tabla libro en data.db, con al menos 10 filas de datos scrapeados.

Salida de ruff check



```
PS C:\Users\Vicente\OneDrive\Desktop\Trabajo3_PC> ruff check .  
All checks passed!
```

Figura 5. Salida ruff check "All checks passed".

Sección de uso de inteligencia artificial

Uso 1

- Herramienta: Claude
- Prompt utilizado: "como puedo evitar que al ejecutar seed.py se inserten libros duplicados en la base de datos?"
- Respuesta obtenida: Se explicó cómo consultar previamente la base de datos utilizando SQLAlchemy y verificar si existía un libro con la misma url_detalle antes de insertar un nuevo registro.
- Cómo se integró: Se incorporó una consulta con `select(Libro).where(Libro.url_detalle == libro["url_detalle"])` para omitir los libros ya almacenados.

Uso 2

- Herramienta: Claude
- Prompt utilizado: "me aparece el error: TypeError: 'generator' object does not support the context manager protocol."
- Respuesta obtenida: Se explicó que la función `get_session()` devolvía un generador mediante `yield` y que no podía utilizarse directamente con `with`, proponiendo devolver una instancia de `Session` o modificar su implementación.
- Cómo se integró: Se modificó la función `get_session()` para que retornara una sesión válida y pudiera utilizarse correctamente en `seed.py` y `queries.py`.

Uso 3

- Herramienta: Claude
- Prompt utilizado: "como puedo recorrer todas las paginas de una categoría utilizando selenium? "
- Respuesta obtenida: Se propuso utilizar un ciclo `while True` y verificar la existencia del botón **Next**, haciendo clic mientras estuviera disponible y finalizando el recorrido cuando ya no existiera.
- Cómo se integró: Se implementó la lógica de paginación dentro del scraper, permitiendo obtener los 1000 libros del sitio.

Uso 4

- Herramienta: Claude
- Prompt utilizado: "como obtengo el numero de estrellas de cada libro desde el html"
- Respuesta obtenida: Se explicó que la valoración se encontraba en la clase CSS del elemento `.star-rating` y que era necesario convertir los valores One, Two, Three, Four y Five a números enteros mediante un diccionario.
- Cómo se integró: Se implementó un diccionario de conversión para almacenar la valoración como un número entero en la base de datos.

Uso 5

- Herramienta: Claude
- Prompt utilizado: "como convierto el precio a decimal"
- Respuesta obtenida: Se sugirió eliminar el símbolo £ utilizando `replace()` y convertir el resultado a float.
- Cómo se integró: El precio fue almacenado como un número decimal, permitiendo realizar consultas y ordenamientos correctamente.

Uso 6

- Herramienta: Claude
- Prompt utilizado: "ayudame a redactar el informe ya hecho para que se vea ordenado y formal"
- Respuesta obtenida: Revisión y reescritura de múltiples secciones del informe: descripción del flujo de scraping, modelo de datos, explicación de las consultas y reflexión final, adaptando el tono a un registro académico formal.
- Cómo se integró: Se revisó cada sección generada para verificar que el contenido técnico fuera preciso y coincidiera con el comportamiento real del scraper. Se realizaron ajustes de redacción donde el texto no reflejaba con exactitud las decisiones tomadas por el equipo, como el manejo de la paginación, la codificación de la valoración por estrellas y los obstáculos encontrados durante el desarrollo.

GITHUB: https://github.com/MayckolO/Trabajo3_PC